1. **Python Programs related to variables and expression showing impact of operators.**

Variables: A variable is a named location in memory used to store data.
In Python, you don't need to declare the type explicitly.
Example:
a = 10
name = "Sam"
Expressions: An expression is a combination of variables, constants, and operators that produces a value.
Example:
result = a + 5
Operators in Python: Operators are used to perform operations on variables and values.
Types of Operators:

Type	Example	Description
Arithmetic	+ - * / % // **	Basic math
Relational	== != > < >= <=	Comparison
Logical	and or not	Combine conditions
Assignment	= += -=	Assign values

Impact of Operators
Operators affect:
- The result value
- The data type
- The logic of the program

Example:
- 5 / 2 = 2.5 (float)
- 5 // 2 = 2 (integer)

Same numbers, different operator gives different result.

•[1]:
```python
# Program to demonstrate variables, expressions and operators
# Variables
a = 10
b = 3
# Arithmetic Operators
print("Addition:", a + b)
print("Subtraction:", a - b)
print("Multiplication:", a * b)
print("Division:", a / b)        # Gives float
print("Floor Division:", a // b) # Removes decimal
print("Modulus:", a % b)
print("Exponent:", a ** b)
# Relational Operators
print("a > b:", a > b)
print("a == b:", a == b)
# Logical Operators
print("a > 5 and b < 5:", (a > 5 and b < 5))
print("not(a > b):", not(a > b))
```

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Floor Division: 3
Modulus: 1
Exponent: 1000
a > b: True
a == b: False
a > 5 and b < 5: True
not(a > b): False
```

2. **Programs related to the conditional statements which shows different impact of If () and ladder if ().**

if Statement:
- The if statement is used to execute a block of code only when a condition is true.
- It checks a condition and runs code once if true.

- Syntax:
  if condition:
  statement

<u>if-elif-else Ladder:</u>

- Used when multiple conditions need to be checked.
- Python checks conditions top to bottom:
  - ➢ The first true condition executes
  - ➢ The rest are skipped
- Syntax:
  if condition1:
  statement1
  elif condition2:
  statement2
  elif condition3:
  statement3
  else:
  statement4

•[2]:
```python
# Program to demonstrate difference between if and if-elif ladder
marks = int(input("Enter marks: "))

print("\nUsing multiple IF statements:")
if marks >= 40:
    print("Pass")
if marks >= 60:
    print("First Class")
if marks >= 75:
    print("Distinction")

print("\nUsing IF-ELIF ladder:")
if marks >= 75:
    print("Distinction")
elif marks >= 60:
    print("First Class")
elif marks >= 40:
    print("Pass")
else:
    print("Fail")
```

```
Enter marks:  56

Using multiple IF statements:
Pass

Using IF-ELIF ladder:
Pass
```

3. **Programs related to the iterative statements which shows different impact of for () and nested for () and while () loop including break and continue statements.**

   Iterative statements are used to execute a block of code repeatedly based on a condition or a sequence.

   - A <u>for loop</u> is used when the number of iterations is known or when iterating over a sequence such as a list, string, or range.
     Syntax:
     for variable in range(start, stop, step):
     statements
   - A <u>nested for loop</u> is a loop inside another loop, and it is used when we need to perform repeated actions within each iteration of the outer loop, such as in patterns or matrices.
     Syntax:
     for i in range():
     for j in range():
     statements

- A while loop is used when the number of iterations is not known in advance and the loop continues execution as long as a given condition is true.
  Syntax:
  while condition:
  statements
- The break statement is used to immediately terminate the loop when a specific condition is met, even if the loop has not completed all iterations.
- The continue statement is used to skip the current iteration and move to the next iteration of the loop.

[3]:
```python
# Program to demonstrate for, nested for, while, break and continue

print("FOR LOOP:")
for i in range(1, 6):
    if i == 3:
        continue   # skip 3
    print(i)

print("\nNESTED FOR LOOP (Pattern):")
for i in range(1, 4):
    for j in range(1, 4):
        print("*", end=" ")
    print()

print("\nWHILE LOOP:")
num = 1
while num <= 5:
    if num == 4:
        break   # stop loop at 4
    print(num)
    num += 1
```

```
FOR LOOP:
1
2
4
5

NESTED FOR LOOP (Pattern):
* * *
* * *
* * *

WHILE LOOP:
1
2
3
```

4. Programs related to the string manipulation using conditional statements and iterations.

- A string in Python is a sequence of characters enclosed in quotes.
- String manipulation refers to operations performed on strings such as counting characters, reversing, checking conditions, and modifying content.
- Conditional statements are used with strings to check specific conditions, such as whether a character is a vowel, digit, or uppercase letter.
- Iterative statements like for and while loops are used to traverse each character of the string and perform operations repeatedly.

•[23]:
```python
# Program to demonstrate string manipulation using iteration and conditions
text = input("Enter a string: ")

vowels = 0
consonants = 0
digits = 0
spaces = 0

for ch in text:
    if ch.isalpha():   # checks if character is alphabet
        if ch.lower() in 'aeiou':
            vowels += 1
        else:
            consonants += 1
    elif ch.isdigit():
        digits += 1
    elif ch.isspace():
        spaces += 1

# Reverse string using loop
reverse = ""
for ch in text:
    reverse = ch + reverse

# Output
print("\nAnalysis of string:")
print("Vowels:", vowels)
print("Consonants:", consonants)
print("Digits:", digits)
print("Spaces:", spaces)
print("Reversed String:", reverse)
```

```
Enter a string:  Apple

Analysis of string:
Vowels: 2
Consonants: 3
Digits: 0
Spaces: 0
Reversed String: elppA
```

5. **Programs related to the string built in functions and their uses in various manners.**
Python provides several built-in functions (methods) for strings that allow easy manipulation and analysis of text. These functions are used to perform operations such as:
   - Changing case (uppercase/lowercase)
   - Searching substrings
   - Replacing text
   - Splitting and joining strings
   - Checking character types

Function	Use
upper()	Converts string to uppercase
lower()	Converts string to lowercase
title()	Converts first letter of each word to capital
strip()	Removes spaces from beginning and end
replace(a, b)	Replaces substring
find()	Finds position of substring
count()	Counts occurrences
split()	Splits string into list
join()	Joins elements into string
isalpha()	Checks if all characters are alphabets
isdigit()	Checks if all characters are digits

•[5]:
```python
# Program to demonstrate string built-in functions
text = input("Enter a string: ")
print("\nOriginal String:", text)

# Case conversion
print("Uppercase:", text.upper())
print("Lowercase:", text.lower())
print("Title Case:", text.title())

# Removing spaces
print("Without spaces:", text.strip())

# Replace word
print("Replace 'a' with '@':", text.replace('a', '@'))

# Searching
print("Position of 'e':", text.find('e'))

# Counting characters
print("Count of 'a':", text.count('a'))

# Splitting string
words = text.split()
print("Split into words:", words)

# Joining string
joined = "-".join(words)
print("Joined with '-':", joined)

# Checking functions
print("Is alphabet only?:", text.isalpha())
print("Is digit only?:", text.isdigit())
```

```
Enter a string:  Python

Original String: Python
Uppercase: PYTHON
Lowercase: python
Title Case: Python
Without spaces: Python
Replace 'a' with '@': Python
Position of 'e': -1
Count of 'a': 0
Split into words: ['Python']
Joined with '-': Python
Is alphabet only?: True
Is digit only?: False
```

6. **Programs related to the function which imply the different effect of passing parameters and arguments.**
A function is a block of code that performs a specific task and can be reused multiple times in a program.

Parameters are the variables listed in the function definition, while arguments are the actual values passed to the function when it is called.
Different ways of passing arguments affect how the function behaves. These include:

- Positional arguments (order matters)
- Keyword arguments (explicit naming)
- Default arguments (predefined values)
- Variable-length arguments (*args)

Syntax:
def function_name(parameters):
statements
return value
Function Call:
function_name(arguments)

•[6]:
```python
# Program to demonstrate different types of arguments
# 1. Positional Arguments
def add(a, b):
    print("Addition:", a + b)

# 2. Default Argument
def greet(name="Guest"):
    print("Hello", name)

# 3. Keyword Arguments
def student(name, age):
    print("Name:", name)
    print("Age:", age)

# 4. Variable-Length Arguments
def total(*numbers):
    sum = 0
    for num in numbers:
        sum += num
    print("Total:", sum)

# Function Calls
print("Positional Arguments:")
add(5, 3)
print("\nDefault Argument:")
greet()
greet("Sam")

print("\nKeyword Arguments:")
student(age=20, name="Alex")
print("\nVariable-Length Arguments:")
total(1, 2, 3)
total(10, 20, 30, 40)
```

```
Positional Arguments:
Addition: 8

Default Argument:
Hello Guest
Hello Sam

Keyword Arguments:
Name: Alex
Age: 20

Variable-Length Arguments:
Total: 6
Total: 100
```

7. **Programs related to the List, tuple, Set, and Dictionary which shows the impact on real implementation.**
Python provides different data structures to store and manage data efficiently.
   - A List is an ordered and mutable collection that allows duplicate values and is used when data needs to be modified frequently.
   - A Tuple is an ordered but immutable collection, meaning its values cannot be changed after creation, making it useful for fixed data.
   - A Set is an unordered collection that does not allow duplicate values, making it ideal for storing unique items.
   - A Dictionary is an unordered collection of key-value pairs, used when data needs to be accessed using a unique key.

[24]:

```python
# List
marks = [85, 90, 78, 92, 70]

# append
marks.append(88)
print("After append:", marks)

# insert
marks.insert(2, 80)
print("After insert:", marks)

# remove
marks.remove(70)
print("After remove:", marks)

# pop
marks.pop()
print("After pop:", marks)

# slicing
print("First 3 marks:", marks[:3])

# sort
marks.sort()
print("Sorted marks:", marks)

# length
print("Total students:", len(marks))
```

```
After append: [85, 90, 78, 92, 70, 88]
After insert: [85, 90, 80, 78, 92, 70, 88]
After remove: [85, 90, 80, 78, 92, 88]
After pop: [85, 90, 80, 78, 92]
First 3 marks: [85, 90, 80]
Sorted marks: [78, 80, 85, 90, 92]
Total students: 5
```

```python
# Tuple
grades = ('A', 'B', 'C', 'A', 'B')

# slicing
print("First 3 grades:", grades[:3])

# length
print("Total grades:", len(grades))

# count occurrence
print("Count of A:", grades.count('A'))

# index
print("Index of C:", grades.index('C'))
```

```
First 3 grades: ('A', 'B', 'C')
Total grades: 5
Count of A: 2
Index of C: 2
```

[26]:
```python
# Set
visitors = {"Amit", "Riya", "Amit", "John", "Riya"}

print("Unique visitors:", visitors)

# add
visitors.add("Neha")
print("After adding:", visitors)

# remove
visitors.remove("John")
print("After remove:", visitors)

# pop (random removal)
visitors.pop()
print("After pop:", visitors)

# length
print("Total unique visitors:", len(visitors))
```

```
Unique visitors: {'John', 'Riya', 'Amit'}
After adding: {'John', 'Neha', 'Riya', 'Amit'}
After remove: {'Neha', 'Riya', 'Amit'}
After pop: {'Riya', 'Amit'}
Total unique visitors: 2
```

[27]:
```python
# Dictionary
student = {
    "name": "Rahul",
    "marks": 85,
    "course": "Data Science"
}

# access
print("Name:", student["name"])

# update
student["marks"] = 90
print("Updated marks:", student)

# add new key
student["city"] = "Pune"
print("After adding city:", student)

# remove
student.pop("course")
print("After removal:", student)

# length
print("Total fields:", len(student))
```

```
Name: Rahul
Updated marks: {'name': 'Rahul', 'marks': 90, 'course': 'Data Science'}
After adding city: {'name': 'Rahul', 'marks': 90, 'course': 'Data Science', 'city': 'Pune'}
After removal: {'name': 'Rahul', 'marks': 90, 'city': 'Pune'}
Total fields: 3
```

8. **Programs related to creation of class and objects which shows the implication of class member function and their accessibility.**

A class is a blueprint used to create objects. It defines properties (variables) and behaviors (functions) of an object.

Syntax:

class ClassName:
def __init__(self):
 # constructor
pass

An object is an instance of a class that contains actual data.

Syntax:

obj = ClassName()

A member function is a function defined inside a class that operates on the data of that class.

In Python, class members can have different levels of accessibility:

- Public members → accessible from anywhere
- Protected members (_) → should not be accessed directly (by convention)
- Private members (__) → cannot be accessed directly outside the class

The accessibility of class members affects data security and control, which is important in real-world applications.

```
•[8]: # Program to demonstrate class, object, member functions and accessibility
class Student:
    def __init__(self, name, marks):
        self.name = name          # Public variable
        self._course = "BSc"      # Protected variable
        self.__marks = marks      # Private variable

    # Public method
    def display(self):
        print("Name:", self.name)
        print("Course:", self._course)
        print("Marks:", self.__marks)

    # Method to access private variable
    def get_marks(self):
        return self.__marks

# Creating object
s1 = Student("Sam", 85)

# Accessing public member
print("Public:", s1.name)

# Accessing protected member (possible but not recommended)
print("Protected:", s1._course)

# Accessing private member using method
print("Private (via method):", s1.get_marks())

print("\nUsing member function:")
s1.display()
```

```
Public: Sam
Protected: BSc
Private (via method): 85

Using member function:
Name: Sam
Course: BSc
Marks: 85
```

9. **Programs related to implementation of inheritance, and polymorphism using various ways of class member and their accessibility.**

Inheritance is a feature of object-oriented programming where one class (child class) acquires the properties and methods of another class (parent class). It promotes code reusability and reduces redundancy.

Types-

1. Single Inheritance
2. Multilevel Inheritance
3. Multiple Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

Polymorphism means "many forms." It allows the same function or method to behave differently depending on the object that calls it. In Python, polymorphism is mainly achieved through:

- Method overriding (same method name, different implementation)
- Method overloading (limited) using default arguments

Accessibility of class members (public, protected, private) also plays an important role in inheritance:

- Public members are accessible everywhere
- Protected members can be accessed in derived classes
- Private members cannot be accessed directly in child classes

```
•[28]: # Program to demonstrate inheritance, polymorphism and accessibility
class Employee:
    def __init__(self, name, salary):
        self.name = name          # public
        self._bonus = 2000        # protected
        self.__salary = salary    # private
    def get_salary(self):         # public method to access private
        return self.__salary
    def total_pay(self):
        return self.__salary + self._bonus
# Inheritance
class Manager(Employee):
    def __init__(self, name, salary, dept):
        super().__init__(name, salary)
        self.dept = dept

    # Polymorphism (method overriding)
    def total_pay(self):
        return self.get_salary() + self._bonus + 5000   # extra allowance
# Another child class
class Developer(Employee):
    def total_pay(self):
        return self.get_salary() + self._bonus + 3000
# Objects
e1 = Employee("Rahul", 30000)
m1 = Manager("Neha", 50000, "IT")
d1 = Developer("Amit", 40000)
# Output
print("Employee Pay:", e1.total_pay())
print("Manager Pay:", m1.total_pay())
print("Developer Pay:", d1.total_pay())
```

```
Employee Pay: 32000
Manager Pay: 57000
Developer Pay: 45000
```

10. Programs related Programs using NumPy and least usable functions of NumPy like modification of chart, plot, and labels. to file handling to show the open, read, write, and append execution.

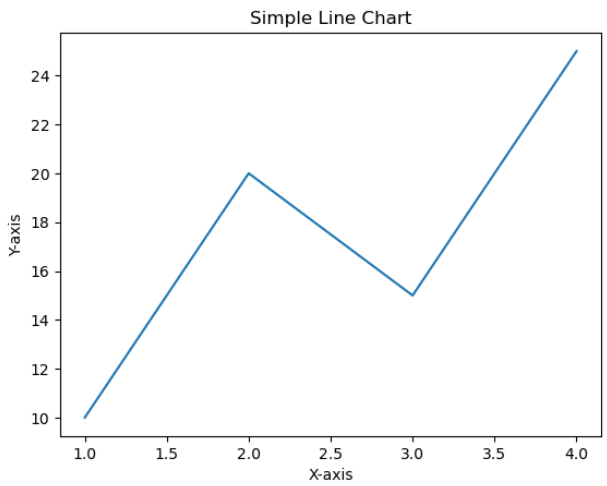
NumPy is a Python library used for numerical computations. It provides support for arrays, mathematical operations, and efficient data processing.

```
[10]: # NumPy array operations

import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print("Array:", arr)
print("Mean:", np.mean(arr))
print("Sum:", np.sum(arr))
print("Square:", arr**2)
```

```
Array: [1 2 3 4 5]
Mean: 3.0
Sum: 15
Square: [ 1  4  9 16 25]
```

Plotting libraries (like Matplotlib) are used to create charts and visualize data. Charts can be modified by adding titles, labels, colors, and grids.

```
•[12]: # Line plot with modifications
import matplotlib.pyplot as plt
x = [1, 2, 3, 4]
y = [10, 20, 15, 25]
plt.plot(x, y)
plt.title("Simple Line Chart")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.show()
```



File handling is used to store and retrieve data permanently. Python provides functions to:

- open() → open/create file
- read() → read file content
- write() → overwrite file
- append() → add data without deleting existing content

[14]:
```
# File handling demonstration
# Write to file
file = open("sample.txt", "w")
file.write("Hello, this is first line.\n")
file.close()

# Read file
file = open("sample.txt", "r")
print("Reading File:")
print(file.read())
file.close()

# Append to file
file = open("sample.txt", "a")
file.write("This is appended line.\n")
file.close()

# Read again
file = open("sample.txt", "r")
print("After Append:")
print(file.read())
file.close()
```

```
Reading File:
Hello, this is first line.

After Append:
Hello, this is first line.
This is appended line.
```

11. Programs related to indexing, slicing, sorting, and matrix manipulation.

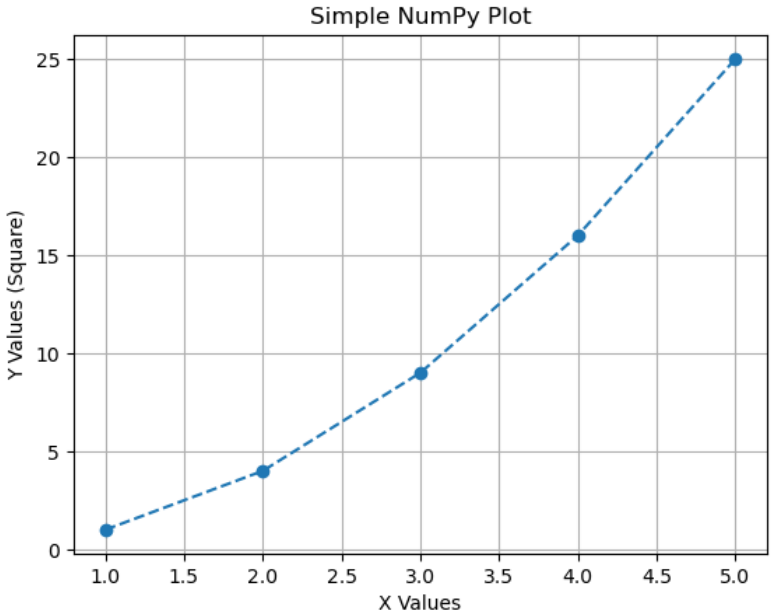
Indexing is used to access individual elements of a sequence using their position.
Slicing is used to extract a portion of a sequence using a range of indices.
Sorting is the process of arranging elements in ascending or descending order.
Matrix manipulation involves operations on 2D arrays such as accessing rows/columns, addition, and transpose.

```python
•[15]: # Program to demonstrate indexing, slicing, sorting, and matrix manipulation
import numpy as np
# 1. Indexing and Slicing
arr = np.array([10, 20, 30, 40, 50])
print("Array:", arr)
print("First element (indexing):", arr[0])
print("Last element:", arr[-1])
print("Sliced array (1 to 3):", arr[1:4])

# 2. Sorting
numbers = [50, 10, 40, 20, 30]
print("\nOriginal List:", numbers)

numbers.sort()
print("Sorted List:", numbers)

# 3. Matrix Manipulation
matrix = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])

print("\nMatrix:\n", matrix)

# Access element
print("Element at (1,1):", matrix[1][1])

# Row and column
print("First Row:", matrix[0])
print("Second Column:", matrix[:, 1])

# Transpose
print("Transpose:\n", matrix.T)

# Matrix addition
matrix2 = np.array([[9, 8, 7],
                    [6, 5, 4],
                    [3, 2, 1]])

print("Matrix Addition:\n", matrix + matrix2)
```

```
Array: [10 20 30 40 50]
First element (indexing): 10
Last element: 50
Sliced array (1 to 3): [20 30 40]

Original List: [50, 10, 40, 20, 30]
Sorted List: [10, 20, 30, 40, 50]

Matrix:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
Element at (1,1): 5
First Row: [1 2 3]
Second Column: [2 5 8]
Transpose:
 [[1 4 7]
 [2 5 8]
 [3 6 9]]
Matrix Addition:
 [[10 10 10]
 [10 10 10]
 [10 10 10]]
```

12. **Programs using NumPy and least usable functions of NumPy like modification of chart, plot, and labels.**
NumPy is used for numerical operations and array manipulation, while plotting libraries like Matplotlib are used to visualize that data.
Less commonly used NumPy functions such as linspace(), reshape(), ravel(), and flatten() help in preparing and transforming data before visualization.
Chart modification includes adding titles, labels, legends, grid, and styles, which improve readability and presentation of data.

```
[29]: import numpy as np
      import matplotlib.pyplot as plt

      # Create data using NumPy
      x = np.arange(1, 6)
      y = x ** 2   # square values

      # Plot
      plt.plot(x, y, marker='o', linestyle='--')

      # Modify chart (labels, title, grid)
      plt.xlabel("X Values")
      plt.ylabel("Y Values (Square)")
      plt.title("Simple NumPy Plot")
      plt.grid(True)

      # Show plot
      plt.show()
```



13. Programs using NumPy to show the classification and clustering of data accessing through storage file.

Classification is the process of assigning data into predefined categories based on certain conditions.
Clustering is the process of grouping similar data points together without predefined labels.
Using NumPy, we can efficiently load and process numerical data stored in files such as .txt or .csv.
File handling allows reading stored datasets and applying classification and clustering logic.

```
•[20]: # Program to demonstrate classification and clustering using NumPy
       import numpy as np
       data = np.loadtxt("C:/Users/Admin/Desktop/Sharvari/MSc/Python Pract/data.txt")
       print("Original Data:", data)

       # 1. Classification
       # Classify into categories: Low (<40), Medium (40-70), High (>70)
       low = data[data < 40]
       medium = data[(data >= 40) & (data <= 70)]
       high = data[data > 70]
       print("\nClassification:")
       print("Low:", low)
       print("Medium:", medium)
       print("High:", high)

       # 2. Clustering (Simple grouping)
       # Cluster into 2 groups based on mean
       mean_value = np.mean(data)
       cluster1 = data[data <= mean_value]
       cluster2 = data[data > mean_value]
       print("\nClustering based on mean =", mean_value)
       print("Cluster 1:", cluster1)
       print("Cluster 2:", cluster2)
```

```
Original Data: [10. 25. 45. 60. 75. 90. 30. 55.]

Classification:
Low: [10. 25. 30.]
Medium: [45. 60. 55.]
High: [75. 90.]

Clustering based on mean = 48.75
Cluster 1: [10. 25. 45. 30.]
Cluster 2: [60. 75. 90. 55.]
```

14. Programs to use the Pandas and related functions to show the data analysis and their relations.

Pandas is a Python library used for data manipulation and analysis. It provides data structures like Series and DataFrame for handling structured data.

Data analysis involves:

- Summarizing data
- Finding patterns
- Identifying relationships between variables

Pandas provides built-in functions to:

- Read data from files
- Perform statistical analysis
- Filter and group data
- Find relationships using correlation

•[30]:
```python
import pandas as pd
data = {
    "Hours": [2, 4, 6, 8, 10],
    "Marks": [50, 60, 70, 80, 90]
}

df = pd.DataFrame(data)

# Display data
print("Dataset:\n", df)
print("\nInformation:")
print(df.info())

# Statistical summary
print("\nSummary Statistics:")
print(df.describe())

# Relationship (Correlation)
correlation = df["Hours"].corr(df["Marks"])
print("\nCorrelation between Hours and Marks:", correlation)
```

```
Dataset:
```

```
Dataset:
    Hours  Marks
0       2     50
1       4     60
2       6     70
3       8     80
4      10     90

Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Hours   5 non-null      int64
 1   Marks   5 non-null      int64
dtypes: int64(2)
memory usage: 212.0 bytes
None

Summary Statistics:
           Hours      Marks
count   5.000000   5.000000
mean    6.000000  70.000000
std     3.162278  15.811388
min     2.000000  50.000000
25%     4.000000  60.000000
50%     6.000000  70.000000
75%     8.000000  80.000000
max    10.000000  90.000000

Correlation between Hours and Marks: 1.0
```

15. Program to show the database connectivity and manipulation of the records like Retrieving record, inserting record, deleting record, updating record, appending record.

Database connectivity refers to establishing a connection between a Python program and a database to perform operations.

Using Python's built-in sqlite3 module, we can create and manage databases easily.

The main database operations are:

- Insert → add new records
- Retrieve (Select) → fetch records
- Update → modify existing records
- Delete → remove records
- Append → add additional records

These operations are collectively called CRUD operations (Create, Read, Update, Delete).

```python
import mysql.connector
# Connect to MySQL Server
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="admin",
    port=3306
)
cursor = conn.cursor()

# 1. Create Database
cursor.execute("CREATE DATABASE IF NOT EXISTS student_db")
print("Database created successfully")

# Use the database
cursor.execute("USE student_db")

# 2. Create Table
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50),
    marks INT
)
""")
print("Table created successfully")

# 3. Insert Record
cursor.execute("INSERT INTO students (name, marks) VALUES (%s, %s)", ("Amit", 85))
conn.commit()
print("Record inserted")

# 5. Retrieve Records
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()
print("\nRecords in table:")
for row in rows:
    print(row)

# 6. Update Record
cursor.execute("UPDATE students SET marks = 95 WHERE name = 'Amit'")
conn.commit()
print("\nRecord updated")

# 7. Delete Record
cursor.execute("DELETE FROM students WHERE name = 'Neha'")
conn.commit()
print("Record deleted")

# Final Display
cursor.execute("SELECT * FROM students")
rows = cursor.fetchall()
print("\nFinal Records:")
for row in rows:
    print(row)

# Close connection
cursor.close()
conn.close()
```

```
Database created successfully
Table created successfully
Record inserted
Record appended

Records in table:
(1, 'Amit', 85)
(2, 'Neha', 90)

Record updated
Record deleted

Final Records:
(1, 'Amit', 95)
```